

Harness Engineering 深度研究報告

執行摘要

「Harness Engineering」在當前 AI 工程語境中，並不是指某一家名為 Harness 的公司，而是指一套為 AI agent 設計**工作環境、控制回路、記憶系統、工具邊界與驗證機制**的新工程方法。它的核心轉向不是「把 prompt 寫得更漂亮」，而是把工程紀律外移到 agent 周圍的**支架系統**：像是 `AGENTS.md`、`CLAUDE.md`、skills、hooks、MCP、測試、lint、結構性約束、可觀測性與背景清理任務。OpenAI 在 2026 年明確把這個工作方式命名為「Harness engineering」，Anthropic 則在更早的脈絡工程與長時程 agent 文章中，把其必要性拆解得非常清楚；Martin Fowler 網站則在 2026 年春季把它升格為可傳授、可複製的方法論。¹

縱向來看，Harness Engineering 不是憑空出現，而是三次工程重心轉移後的結果。第一階段是 prompt engineering，重點在怎麼對模型發指令；第二階段是 context engineering，重點在怎麼管理有限 context window 內的資訊配置；第三階段才是 harness engineering，重點改成**怎麼設計整個 agent 執行系統**，讓模型在多步驟、多工具、長時間、多代理的工作中可控、可驗證、可持續。Anthropic 明確把 context engineering 定義為 prompt engineering 的自然延伸，而 OpenAI 則進一步指出，當開發流程變成「人類定義意圖、agent 執行、系統持續驗證與修正」時，真正稀缺的資源已經不是 token，而是**人類的時間與注意力**。²

橫向來看，2026 年最有代表性的競爭，並不是誰「有沒有 agent」，而是誰把 Harness Engineering **產品化得更完整**。OpenAI Codex 最像概念原型與參考實作；Anthropic Claude Code 在 hooks、session 設計、工具安全隔離上最強；Cursor 在 IDE 原生體驗、rules、skills、subagents、background/cloud agents 上最成熟；GitHub Copilot 的優勢在 repo-native 與 PR / issue / Actions 整體流程；Devin Cloud 與 Devin Desktop 則把「多代理指揮中樞」推到最前。這個競爭格局顯示，差異化正從「單一模型能力」快速移向「環境設計、控制面、治理性與成本可預測性」。³

我的核心判斷是：在未來一至兩年內，Harness Engineering 會從前沿團隊的內部 best practice，變成主流軟體工程平台的**預設能力層**。原因很直接：多家產品已開始支援多模型、第三方 agent、MCP、skills、rules、cloud/background execution，模型本身正在趨於可替換，而真正難以替換的，是與企業程式庫、票務系統、觀測資料、政策控制、成本治理和審查流整合的「外部系統」。也就是說，未來的護城河不會只在模型，而會更深地落在 harness。這是本報告最重要的結論。⁴

研究界定與分析框架

本報告把「Harness Engineering」界定為**AI coding agents 的環境工程學**。之所以採用這個界定，是因為目前可取得的高可信來源——OpenAI、Anthropic 與 Martin Fowler——都在 2025 年底到 2026 年間，用這個詞或同義的描述，討論 agent 周圍的控制、上下文、驗證與治理結構，而不是某一個單獨模型或單一 prompt 技巧。⁵

因此，橫向分析時，我不把比較對象設成抽象概念，例如「prompt engineering」或「context engineering」本身，而是改成比較**目前把 Harness Engineering 商業化、工具化、工作流化的主要產品與平台**。這樣做比較符合你要求的欄位：技術路線、產品形態、商業模式、目標用戶、用戶體驗、優缺點、生態位勢與趨勢判斷。⁶

還有一個重要說明：Harness Engineering 作為概念，**沒有公司式的 founding date**；可取得的一手資料也**沒有明確標出唯一且無爭議的 first coinage date**。就本次研究所能確認的材料而言，最早可追溯的官方前身，是 Anthropic 在 2025 年 11 月公開討論 “effective harnesses”；最早明確以 “Harness engineering” 命名並對外系統化說明的高可信一手來源，是 OpenAI 在 2026 年 2 月 11 日的文章。至於「誰最先發明了這個名詞」，目前仍屬**未完全指定事項**；Martin Fowler 站上的作者甚至明說，該詞很可能是在 OpenAI 文章發布後才快速成形，也可能受 Mitchell Hashimoto 2026 年 2 月初「Engineer the Harness」一文的語感啟發。 ⁷

縱向敘事

如果把 2022 到 2026 這段歷史看成一條敘事線，那它其實不是「模型越來越強」的單線故事，而是**工程責任一再外移**的故事。

最早，大家相信問題主要出在「怎麼問模型」。Prompt engineering 因而成為第一代顯學。OpenAI 的官方指南與 AWS 的說明都把 prompt engineering 定義為優化輸入指令以獲得更好輸出的技藝；這在單輪生成、分類或簡單內容生產上確實有效。但當任務開始需要跨檔案、跨工具、跨多輪對話，單一 prompt 很快遇到極限。它能表達意圖，卻無法充當完整作業系統。 ⁸

到了 2025 年，前沿團隊逐漸看清新的瓶頸：不是你會不會寫 prompt，而是模型**在那一刻究竟看到了什麼**。Anthropic 在 2025 年 9 月把這個轉向正式命名為 context engineering，並明確說它是 prompt engineering 的自然延伸。關鍵原因有兩個。第一，agent 是多輪運行的，context 會持續累積與污染；第二，context 是有限資源，加入更多 token 並不等於更好，反而可能讓模型失焦。這一步的決策邏輯很清楚：若要讓 agent 變成可長時間工作的「系統」，就不能只調 prompt，而要管理整個 context 狀態。 ⁹

但 context engineering 仍然不夠。因為真正的工作不是一次推理，而是**跨多個 context window 的長時程任務**。Anthropic 在 2025 年 11 月的 “Effective harnesses for long-running agents” 文章，把這個問題說得非常具體：每個新 session 都像是換班工程師，若沒有記錄與交接，前一班做過的事就等於不存在。因此他們設計了雙代理結構——一個 initializer agent 負責初始化與鋪設環境，一個 coding agent 負責每個 session 的增量推進，並要求後者留下清晰 artifacts 交接給下一輪。這是個關鍵轉折：工程團隊不再只是教模型「怎麼做」，而是開始設計**讓下一輪 agent 能接得上的環境**。 ¹⁰

真正讓這個思路爆發性進入公眾視野的，是 OpenAI 公布的 Codex 內部實驗。根據 OpenAI 2026 年 2 月文章，他們在 2025 年 8 月下旬向一個空白 repo 提交了第一個 commit，接著用五個月時間，用 Codex 為一個真實產品寫出約百萬行程式碼、基礎設施、測試、CI、文件、儀表板與內部工具；整個過程中，小團隊大約合併了 1,500 個 PR，OpenAI 估計速度約為手寫程式碼的十分之一時間成本。更重要的不是效率數字，而是角色改寫：人類不再主要寫 code，而是**設計環境、指定意圖、鋪設 feedback loops**。OpenAI 用一句極有代表性的話總結這個重心變化：**Humans steer. Agents execute.** ¹¹

這篇文章披露的技術選擇，也構成了 Harness Engineering 的第一個清晰範本。OpenAI 很早就發現「一個超長 AGENTS.md」會失敗，因為 context 稀缺、重要性失焦、規則會腐爛，而且不利機械驗證。所以他們改把 AGENTS.md 當作 table of contents，把真正的知識庫放入 versioned docs/ 結構中，讓 codebase 本身成為 system of record。接著，他們讓 agent 能讀到 UI、瀏覽器、logs、metrics、traces，並用 Chrome DevTools、LogQL、PromQL 等信號去驗證自己的修改；再往上一層，用 custom linters、structural tests、layered architecture 與「taste invariants」把可維持的工程標準變成機械邊界；最後，再用背景清理任務去掃描偏差、更新品質分數、開出小型 refactor PR，像垃圾回收一樣持續壓低熵。每一個決策背後的邏輯其實都一致：**當 agent throughput 超過 human attention 時，不能再靠人工口頭糾偏，必須把糾偏本身變成系統能力**。 ¹²

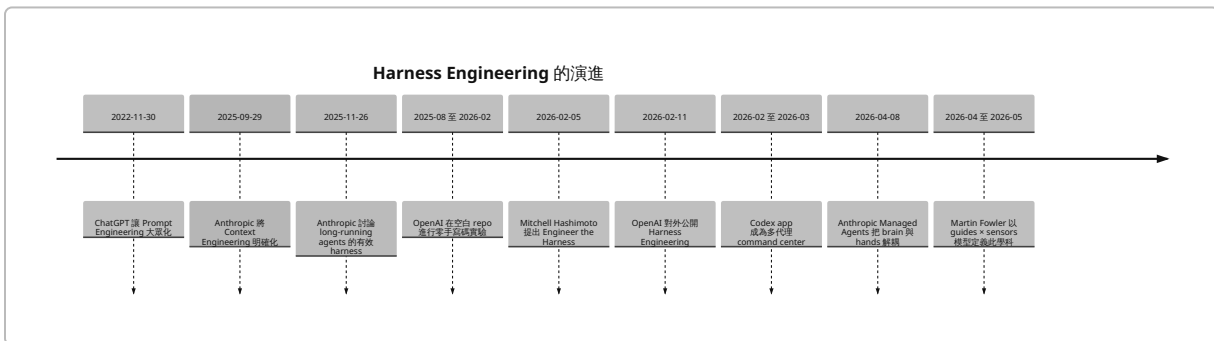
隨後，這套做法被更系統化地抽象為方法論。2026 年春季，Martin Fowler 網站把 Harness Engineering 定義為圍繞 coding agent 的一組 **guides** 與 **sensors**。Guides 是前饋控制：在 agent 行動前，提升其第一次就做對的機率；Sensors 是回饋控制：在 agent 行動後，讓它能自我修正。更進一步，Birgitta Böckeler 還把二者劃分為 computational 與 inferential 兩類。這個框架之所以重要，不只因為它整理了概念，更因為它把過去零散的檔案、規則、lint、測試、AI review、可觀測性、治理政策放到同一張圖上，讓團隊可以討論「我們的 harness 缺哪一象限」，而不再只談 model prompt 好不好。 ¹³

同一時間，Anthropic 也在往更深的系統層推進。2026 年 4 月的 Managed Agents 文章裡，他們把「brain」與「hands」解耦，讓 session log 成為 context window 之外可查詢、可恢復的長期上下文物件，並把 OAuth 憑證留在 sandbox 外，由 vault 與 MCP proxy 代理，避免 agent 直接接觸權杖。這告訴我們一件事：Harness Engineering 到了 2026 年，已經不只是生產力話術，而是**安全架構與執行邊界工程**。 ¹⁴

時間序里程碑表

日期	里程碑	初始形態	為何這一步會出現
2022-11-30	ChatGPT 公開，prompt engineering 成為主流工程關注點	以對話與指令優化為中心的互動界面	當時主要問題是讓模型在單輪任務上更可用，因此工程重心集中在 prompt wording 與格式化輸入。 ¹⁵
2025-09-29	Anthropic 發表 context engineering	「管理 token 組態」的方法論	因為 agent 已跨多輪與多工具運作，prompt 不再足夠；context 被明確認定是有限且需持續策展的資源。 ⁹
2025-11-26	Anthropic 發表 long-running agents 的 effective harnesses	initializer agent + coding agent + artifacts 交接	理由是 context window 不能承載長時程歷史，必須用外部 artifacts 與角色分工讓 agent 跨 session 延續工作。 ¹⁰
2025-08 下旬	OpenAI 在空白 repo 啟動「零手寫碼」內部實驗	一個由 Codex 主導生成的全新 codebase	這是刻意設下的強制條件，目的是逼團隊建出真正能放大工程速度的環境，而不是靠人類補洞。 ¹¹
2026-02-05	Mitchell Hashimoto 提出「Engineer the Harness」的採用階段	實務工作流語言	顯示一線工程師已把焦點從聊天式使用，移到如何為 agent 設計穩定工作環境。 ¹⁶
2026-02-11	OpenAI 對外公開“Harness engineering”	不是產品，而是方法論與內部實驗總結	這是本次研究能確認的最早高可信公開命名；初始形態是支援 Codex 的 repo 結構、驗證回路、觀測與清理系統。 ¹⁷

日期	里程碑	初始形態	為何這一步會出現
2026-02-02 起， 2026-03-04 更新	Codex app 上線並擴充到 Windows	多 agent、平行執行、worktree 與 Git 的 command center	之所以需要新產品形態，是因為長任務與多代理協作不再適合只靠單一 CLI 視窗管理。 18
2026-04-08	Anthropic 發表 Managed Agents	brain/hands 解耦、session log 外部化、憑證隔離	當 agent 走向長時程與企業環境後，安全邊界與可恢復上下文比單次回覆品質更重要。 14
2026-04 至 2026-05	Martin Fowler 網站把模型正式化為 guides × sensors，並延伸到 maintainability sensors	可教學、可評估、可模板化的方法論	當越來越多團隊都在做類似事情時，需要一套通用語彙來複用與審查 harness 設計。 13



這條縱向脈絡的真正含義是：Harness Engineering 不是單純技術名詞更替，而是**軟體工程控制論的回歸**。以前這些控制是靠資深工程師腦中的 tacit knowledge 與團隊習慣維持；現在，控制被外顯化、檔案化、工具化、版本化，並且直接供 agent 消費。這也是它之所以值得被看成新學科，而不是 prompt engineering 2.0 的原因。
19

橫向競局

在 2026 年當下，Harness Engineering 最合適的橫向比較單位，不是抽象概念，而是**把這個方法落成產品的主要平台**。下面的表格把目前最具代表性的五個前沿實作放在同一張圖上。

競品比較表

平台	技術路線	產品形態	商業模式	目標用戶	用戶視角	生態位勢與趨勢判斷
OpenAI Codex	AGENTS.md、skills、subagents、worktrees、in-app browser、cloud agent、Git 與 automations；OpenAI 也用同樣原理做出 repo knowledge system、custom linters、observability 與 garbage collection。 ²⁰	CLI、桌面 app、雲端、行動端延伸；明確主打多線程與平行 agent。 ²¹	包含在 ChatGPT Plus / Pro / Business / Edu / Enterprise 中，另有工作場景與企業化路線。 ²²	需要平行任務、跨裝置監控、以 agent 為主體工作的工程師與技術團隊。 ²³	Product Hunt 評價偏正面：用戶特別稱讚它會先讀真實 codebase、在 sandbox 中測試後再交付、多檔案任務可信度較高；缺點則是部分用戶覺得執行較慢、進度回饋偏弱、大 repo 時仍可能丟失脈絡。 ²⁴	生態位勢： 最接近「原生 Harness Engineering 參考實作」。 趨勢： 若 OpenAI 持續把 app、cloud、mobile、security 與 agent review 串成一體，Codex 很可能成為 reference stack；風險是年輕產品的公開評價樣本仍偏薄。 ²⁵

平台	技術路線	產品形態	商業模式	目標用戶	用戶視角	生態位勢與趨勢判斷
Anthropic Claude Code	CLAUDE.md、auto memory、path-scoped rules、hooks、skills、subagents、MCP；Managed Agents 進一步把 session 放到 context window 外、把憑證隔離在 sandbox 外。 26	Terminal、VS Code、JetBrains、桌面、Web、Slack。 27	個人 Pro / Max、Team、Enterprise，另可走 API token consumption；Claude Code 文件顯示企業平均每位活躍開發者每月約 150–250 美元。 28	終端機工作流強、對控制與安全邊界敏感、需要長時間的 engineering 團隊。 29	G2 把它描述為可讀整個 codebase、可編輯檔案、跑命令並與工具整合的 agentic coding tool；但公開可見的長篇評論樣本仍不如 Cursor / Copilot 豐富。可觀察到的負面訊號主要來自成本波動與服務可用性：文件強調按 token 計價，狀態頁也顯示 2026 年 6 月初會有 Opus / Sonnet 服務異常。 30	生態位勢：目前最強的「可控性導向」實作之一。趨勢：在企業安全、長時程 session 和系統級控制上會持續有優勢；風險是成本可預測性與單一供應商依賴。 31

平台	技術路線	產品形態	商業模式	目標用戶	用戶視角	生態位勢與趨勢判斷
Cursor	rules、skills、MCP、hooks、background/cloud agents、subagents、Bugbot、team marketplace；官方 changelog 明言這些是 agent harness improvements。 ³²	AI-native IDE，延伸到 CLI 與 cloud agent；強烈 VS Code 遷移友好。 ³³	個人 Pro 20 美元/月；Teams 40 美元/席/月；Enterprise 客製。 ³⁴	想把 agent 直接嵌進編輯器、以互動式編碼為主的開發者與團隊。 ³⁵	G2 評價很集中：優點是介面像 VS Code、上手快、多檔修改與 chat + inline workflow 高效，能明顯節省重複性工作；缺點是複雜任務上回答有時不穩、較大 codebase 會變慢、價格彈性不足。 ³⁶	生態位勢： IDE-native 最完整。 趨勢： 若要在個人開發者與小團隊端建立習慣，Cursor 目前位置非常強；主要風險在於企業治理與成本控制是否能跟上它的產品節奏。 ³⁷

平台	技術路線	產品形態	商業模式	目標用戶	用戶視角	生態位勢與趨勢判斷
GitHub Copilot	cloud agent 在 GitHub Actions ephemeral environment 中完成研究、規劃、改碼、PR；另有 IDE agent mode、custom agents、MCP、agent skills、code review。 38	GitHub.com、VS Code、Visual Studio、JetBrains、CLI；最強是直接嵌進 repo / issue / PR / review / Actions 流程。 39	Pro 10 美元/月、Pro+ 39、美金、Max 100，美國時間 2026-06-01 起全面轉為 usage-based billing。 40	已深度使用 GitHub 工作流、希望把 agent 放進 PR 與 issue 流程的團隊。 41	G2 與 TrustRadius 的共識是：即時補全、樣板程式碼與 IDE 整合很強，能減少大量重複工作；典型缺點則是 edge cases 不夠穩、在大型 codebase 上上下文理解有限、agent mode 可能偏慢。2026 年 6 月 usage-based billing 上線後，又出現明顯價格反彈。 42	生態位勢： repo-native 與協作治理最強。 趨勢： 只要 GitHub 仍是主開發平面，它就有天然黏性；但計費 shock 與平台中心化也可能促使高用量團隊尋求替代。 43

平台	技術路線	產品形態	商業模式	目標用戶	用戶視角	生態位勢與趨勢判斷
Devin Cloud + Devin Desktop	多代理 command center、shared context、Memories & Rules、Workflows、AGENTS.md、hooks、MCP；Windsurf 已在 2026 年正式改名為 Devin Desktop。 ⁴⁴	Cloud agent + desktop IDE / command center 雙棲；官方把它定位成「所有 agent 的家」。 ⁴⁵	Free、Pro 20、美金/月、Max 200、美金/月、Teams 80 + 40/全職席位、Enterprise 客製。 ⁴⁶	多 repo、長時程、多代理協作的工程團隊；偏向「把 agent 當數位員工陣列」的團隊。 ⁴⁵	G2 對原 Windsurf 的評價偏向「易用、AI 整合好、能提升效率」，但也指出大專案下流暢度較差；TrustRadius 則認為 ROI 顯著，但指出它更適合中高階使用者，對初學者不夠直觀。 ⁴⁷	生態位勢： 最強調「多代理指揮」與 shared workspace。 趨勢： 若 Devin 與 Windsurf 的整合成功，它可能成為 multi-agent ops 的代表；風險是品牌遷移與產品整合複雜度。 ⁴⁸

這張表背後有一個很值得注意的橫向結論：**大家都開始把模型當作可插拔層，而把 harness 當作真正差異化層。**Cursor 強調 frontier models 與 MCP / skills / hooks；GitHub Copilot 甚至直接把第三方 agents（Claude Code、Codex）納入方案；Claude Code 支援第三方 providers；Devin 方案則提出支援 major model providers。這意味著市場競爭正在從「你是不是有最強模型」，轉成「你能不能把不同模型放進更可用、更可審計、更低摩擦的工作環境中」。 ⁴⁹

Harness Engineering 的短版 SWOT

Strengths：它把人類工程師原本隱性的知識——規範、架構、審查標準、除錯路徑——外顯成 agent 可消費的系統，從而顯著壓縮人類注意力成本。OpenAI 甚至在內部實驗中報告了約 1/10 的開發時間量級；Anthropic 與 Martin Fowler 的模型則說明，guides 與 sensors 能提高第一次做對的機率，並形成自我修正回路。 ⁵

Weaknesses：它的前置工程並不輕。要把 repo 結構、規則檔、hooks、skills、測試、觀測、政策控制與成本治理都做好，本身就是一個平台工程問題。OpenAI 也坦承，目前仍不知道全 agent codebase 經年演化後的架構一致性會如何；Anthropic 也持續強調 context 是稀缺資源，代表 harness 不是打造一次就結束，而是要持續維護。 ⁵⁰

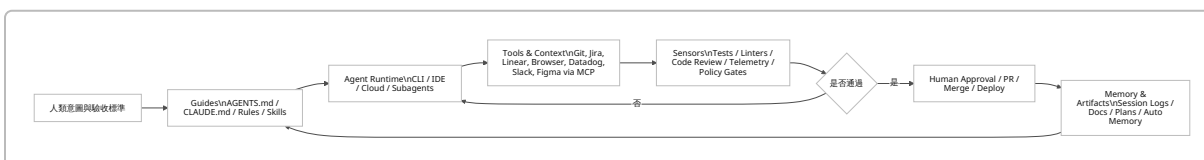
Opportunities：開放格式與協定正在出現。AGENTS.md 已成為多平台共通語言，MCP 正成為工具接入標準；這讓組織有機會把「agent 工作環境」做成可攜、可治理、可組織擴散的資產，而不是綁死在某一家 editor 的私有設定檔裡。 ⁵¹

Threats：三種風險最現實。第一是**安全**：prompt injection、權杖暴露、sandbox 邊界失守，已經迫使 Anthropic 重新設計 brain/hands separation。第二是**成本與計費衝擊**：GitHub 已轉 usage-based billing，

Anthropic 與 Cursor 也都強調 consumption / overage 機制。第三是**服務與供應商依賴**：近端 outage、平台計費變更或產品策略調整，都會直接撞上研發流程。 52

交叉判讀與未來走向

把縱向歷史與橫向競局放在一起，現在的局勢其實非常鮮明：Harness Engineering 正從「前沿研究團隊的內部工藝」，走向「軟體交付平台的主競爭層」。它之所以會跨過那道門檻，不是因為某一家產品一次性做對了所有東西，而是因為市場開始出現**共同且可辨識的結構收斂**：repo-level instructions、memory、skills、hooks、MCP、background / cloud agents、多代理、AI code review、usage analytics、shared context、安全沙箱。當這些結構同時在 OpenAI、Anthropic、Cursor、GitHub、Devin 生態裡出現時，就很難再把它看成零散 feature；它更像一個正在形成中的標準工程層。 53



這也讓「現在 Harness Engineering 的位置是什麼」可以用一句話概括：**它已經從 agent 的輔助層，變成 agent 的生產系統層**。Prompt 還在，context 還在，但它們越來越像 CPU 與 RAM 的配置問題；真正決定團隊能否把 AI 編碼轉成穩定產出的，是外層的運行框架。Anthropic 說得最直白：session 並不等於 context window；OpenAI 說得最直白：真正難的是 scaffold，而不是 code generation 本身。 54

證據導向的預測

第一個預測：模型會愈來愈像可替換零件，harness 會變成主要護城河。

這不是口號，而是從產品設計可直接觀察到的：Cursor 主打 frontier models 接入；GitHub Copilot 方案中已可用第三方 agents 與 model selection；Claude Code 支援 third-party providers；Devin 方案則強調對 major model providers 的一級支援。這些訊號一致地指向同一件事：前沿模型仍重要，但在產品層面，它們正在變成「可被更大環境調度的部件」。 49

第二個預測：repo-native instruction formats 會收斂，而 AGENTS.md 會成為跨平台最有可能的最低共識。

Codex 把 AGENTS.md 定位為 custom instructions 的自動載入口；Cursor 文件明確把 AGENTS.md 視為 persistent instruction 系統的一部分；Windsurf / Devin Desktop 也已支援 directory-scoped AGENTS.md；Anthropic 雖以 CLAUDE.md 為主，但文件也直接列出對 AGENTS.md 的支援與對照。這代表 instruction portability 正在從「nice to have」變成實際需求。 55

第三個預測：長時程與多代理會從高階功能變成標配。

Codex app 以平行 threads/worktrees 為核心賣點；GitHub cloud agent 把背景工作放在 GitHub Actions 中；Cursor 已把 background agents、subagents、cloud agents 納入主線體驗；Devin Desktop 更直接把自己定位成「管理所有 agents 的 command center」；Anthropic 則在 Managed Agents 中把跨 session 的資料結構正式化。這種多方同時發生的產品趨勢，通常不是短期實驗，而是平台方向。 56

第四個預測：採購決策會從「誰最聰明」轉向「誰最可治理」。

企業真實買單的痛點，已經很明顯地落在支出、審計、隱私、沙箱與流程整合上。GitHub 已全面切到 AI credits 計價；Cursor 已把 usage analytics、privacy mode、MCP access controls 放進企業方案；Anthropic 提供 spend

limits、usage breakdown 與 enterprise admin；Devin 方案中則有 team analytics、dedicated deployment 與 enterprise admin control。當治理能力進方案首頁，代表市場正在把它當一級需求。 57

戰略建議

技術面

如果你是研究團隊、平台團隊，或正在設計自己的 agent workflow，最值得優先做的不是追逐更多 prompt 技巧，而是建立**雙層控制系統**：先用 guides 把 repo-level knowledge、path-specific rules、skills、architecture docs 固化；再用 sensors 把 lint、tests、code review、telemetry 變成 agent 可理解的回饋訊號。OpenAI 的經驗特別值得學：把錯誤訊息寫成 agent 可自修的說明，把 documentation 當系統紀錄，而不是把規則塞成單一肥大指令檔。 58

產品面

下一代優秀工具不該只是一個「更會寫 code 的聊天框」，而應該是一個**可以觀測、可回放、可審計、可計費、可恢復**的 command center。從 Codex app、Devin Desktop、GitHub cloud agent 的共同方向看，未來更有價值的功能包括：agent diff provenance、跨代理 shared context、成本儀表板、失敗回放、風險等級審查、approval reviewer、自動修復回路與可觀測性回灌。也就是說，產品的主體不應再只是模型，而是「代理操作系統」。 59

Go-to-market 面

市場訊息也該改寫。對企業買家而言，最有說服力的賣點不會是「比別家更聰明」，而是「能把人類審查負擔減少多少、把 backlog 問題處理掉多少、把每位開發者的 agent 成本控制在何範圍、把 policy violation 降低多少」。Anthropic 在成本文件中已經給出每位開發者每日與每月成本區間；GitHub 與 Cursor 也都把 usage reporting 與 analytics 放到核心方案說明。這意味著 GTM 應該賣的是**工程經濟學**，而不只是模型魔法。 60

合作夥伴面

最值得投資的不是封閉工具鏈，而是**MCP 與工作系統夥伴**。原因很務實：真正決定 agent 有沒有價值的，不是它能不能在一個玩具 repo 寫出 code，而是它能不能吃到 Jira / Linear / Slack / Datadog / Sentry / Figma / Vercel / GitHub 上的真實上下文，並在安全邊界內做事。Windsurf / Devin Desktop、GitHub Copilot、Claude Code、Codex 全都已在這條路上。誰掌握最多高價值的 context surfaces，誰就比較可能贏。 61

開放問題與關鍵來源

開放問題與限制

第一，“**Harness Engineering**”的**精確首創日期與首創者仍未完全指定**。本報告能確認的高可信事實，是 Anthropic 在 2025 年 11 月公開談 harnesses，OpenAI 在 2026 年 2 月 11 日正式公開使用 “Harness engineering” 一詞；再往前的語源與民間首發，目前仍是半開放問題。 62

第二，OpenAI 與 Anthropic 公布的部分內部成效，例如百萬行程式碼、1/10 時間量級、企業平均成本等，都屬於**公司自陳或文件自陳數據**。它們非常有研究價值，但不應被當成跨所有團隊、所有工具、所有程式碼庫可直接複製的普適結論。OpenAI 自己也明言，Codex 的端到端自治高度依賴特定 repo 結構與同等程度的投資。 63

第三，**公開用戶評論樣本不均衡**。Cursor、GitHub Copilot、Windsurf 的公開 review surface 相對豐富；Codex、Claude Code、Devin 這類新一代 agent 產品則較多依賴較新的 Product Hunt、G2 列表、論壇或官方方案

例，公開可見的「長時間、成規模、可交叉驗證」評論仍然不足。這代表橫向 UX 比較應被視為現階段最佳可得估計，而不是終局排名。 64

主要來源

- **OpenAI — Harness engineering: leveraging Codex in an agent-first world**：本報告對 Harness Engineering 公開命名、Codex 實驗、repo knowledge、observability、architecture constraints、garbage collection 的主要一手來源。 65
- **Anthropic — Effective context engineering for AI agents**：提供從 prompt 到 context 的理論躍遷與 context 稀缺性的核心定義。 9
- **Anthropic — Effective harnesses for long-running agents**：提供 harness 作為跨 session、長時程 agent 支架的最早一手前身。 10
- **Anthropic — Scaling Managed Agents**：提供 brain/hands 解耦、session 外部化與憑證隔離的安全架構視角。 14
- **Martin Fowler — Harness engineering for coding agent users / first thoughts / maintainability sensors**：把概念整理成 guides × sensors、computational × inferential 的可操作框架。 13
- **OpenAI Codex / Anthropic Claude Code / Cursor / GitHub Copilot / Devin Desktop 官方文件與價格頁**：用於當前橫向競品技術路線、產品形態與商業模式比較。 66
- **G2 / TrustRadius / Product Hunt 公開評論頁**：用於補足真實用戶視角、UX 優缺點與價格壓力。 67

總結一句：**Harness Engineering** 的誕生，標誌著 **AI coding** 從「模型互動技巧」進入「代理生產系統工程」階段。下一個時代的贏家，不會只是最會回答問題的模型供應商，而會是最能把知識、工具、政策、驗證、記憶與成本，整合成一個可持續工作環境的系統設計者。 68

1 5 11 12 17 19 25 50 58 63 65 68 **Harness engineering: leveraging Codex in an agent-first world | OpenAI**

<https://openai.com/index/harness-engineering/>

2 9 **Effective context engineering for AI agents \ Anthropic**

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

3 6 22 59 66 **Codex app**

https://developers.openai.com/codex/app?utm_source=chatgpt.com

4 27 29 **Overview - Claude Code Docs**

<https://docs.anthropic.com/en/docs/claude-code/overview>

7 10 62 **Effective harnesses for long-running agents \ Anthropic**

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

8 **Prompt engineering | OpenAI API**

https://developers.openai.com/api/docs/guides/prompt-engineering?utm_source=chatgpt.com

13 **Harness engineering for coding agent users**

https://martinfowler.com/articles/harness-engineering.html?utm_source=chatgpt.com

14 31 52 54 **Scaling Managed Agents: Decoupling the brain from the hands \ Anthropic**

<https://www.anthropic.com/engineering/managed-agents>

15 **Introducing ChatGPT**

https://openai.com/index/chatgpt?utm_source=chatgpt.com

16 **My AI Adoption Journey**

https://mitchellh.com/writing/my-ai-adoption-journey?utm_source=chatgpt.com

18 23 56 **Introducing the Codex app**

https://openai.com/index/introducing-the-codex-app?utm_source=chatgpt.com

20 51 55 **Custom instructions with AGENTS.md – Codex**

https://developers.openai.com/codex/guides/agents-md?utm_source=chatgpt.com

21 **Codex CLI**

https://developers.openai.com/codex/cli?utm_source=chatgpt.com

24 **Codex 3.0 by OpenAI**

https://www.producthunt.com/products/codex-3-0-by-openai?utm_source=chatgpt.com

26 **How Claude remembers your project - Claude Code Docs**

<https://docs.anthropic.com/en/docs/claude-code/memory>

28 **Plans & Pricing | Claude by Anthropic**

https://claude.com/pricing?utm_source=chatgpt.com

30 **Claude Code Reviews 2026: Details, Pricing, & Features**

https://www.g2.com/products/anthropic-claude-code/reviews?utm_source=chatgpt.com

32 34 37 49 **Cursor · Pricing**

<https://cursor.com/pricing>

33 **Cursor CLI — Run Agents in Terminal, GitHub Actions and ...**

https://cursor.com/cli?utm_source=chatgpt.com

35 **Best practices for coding with agents · Cursor**

<https://cursor.com/blog/agent-best-practices>

36 64 67 **Cursor Reviews 2026: Details, Pricing, & Features**

https://www.g2.com/products/cursor/reviews?utm_source=chatgpt.com

38 41 43 **About GitHub Copilot cloud agent - GitHub Docs**

<https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent>

39 40 **GitHub Copilot · Plans & pricing · GitHub**

<https://github.com/features/copilot/plans>

42 **GitHub Copilot Reviews 2026: Details, Pricing, & Features**

https://www.g2.com/products/github-copilot/reviews?utm_source=chatgpt.com

44 48 61 **Devin Desktop | Devin**

<https://windsurf.com/cascade>

45 **Devin | The AI Software Engineer**

https://devin.ai/?utm_source=chatgpt.com

46 **Plans and Pricing | Devin**

<https://windsurf.com/pricing>

47 **Windsurf Reviews 2026: Details, Pricing, & Features**

https://www.g2.com/products/exafunction-windsurf/reviews?utm_source=chatgpt.com

53 **Customization – Codex**

https://developers.openai.com/codex/concepts/customization?utm_source=chatgpt.com

57 **GitHub Copilot is moving to usage-based billing - The GitHub Blog**

<https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/>

60 **Manage costs effectively - Claude Code Docs**

<https://docs.anthropic.com/en/docs/claude-code/costs>